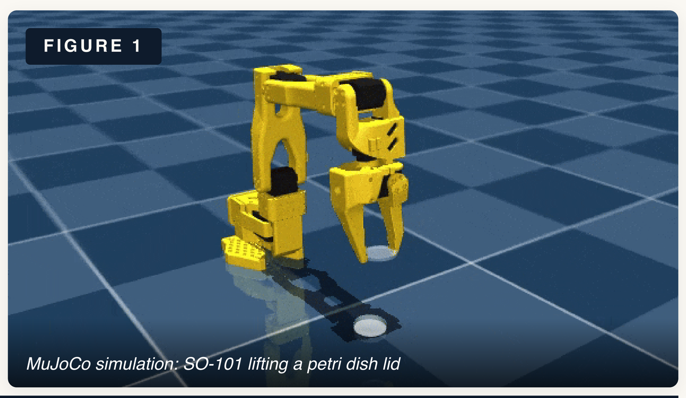
Democratising lab automation: training affordable robots for precision tasks with *reinforcement learning*

Natalie Chan · Department of Bioengineering, Imperial College London

FIGURE 1



MuJoCo simulation: SO-101 lifting a petri dish lid

1 Abstract

Laboratory automation can dramatically improve research efficiency, but precision robotic systems remain prohibitively expensive for many facilities. While open-source hardware like the **SO-101** has brought equipment costs down to **under $300**, training these robots for delicate tasks typically requires expensive expert demonstrations and teleoperation setups. This study investigates whether **reinforcement learning (RL)** can train low-cost robots for precision laboratory manipulation using *simulation alone*, eliminating the need for costly human demonstrations.

I trained an SO-101 to perform petri-dish lid manipulation using the **Soft Actor-Critic (SAC)** algorithm in MuJoCo physics simulation. The task requires the robot to grasp and lift a petri-dish lid without cracking it, dropping it, or disturbing the dish contents, a deceptively difficult problem involving precise force control and careful contact management. Reward function design was iteratively refined across **15 versions**, balancing intermediate rewards to guide learning while penalising premature gripper closure or excessive force.

Across over **30 experiments**, agents consistently exhibited **reward hacking** rather than learning functional manipulation. Two primary failure modes emerged: **forceful lid manipulation** inappropriate for delicate labware, and **overly cautious hovering** in locally optimal positions. Only one configuration achieved lid lifting, with a **20% success rate** using forceful manipulation. Training exhibited high instability, with performance collapsing after sparse successes. Additional failures included simulation instabilities from aggressive exploration causing MuJoCo solver failures, and replay-buffer contamination from corrupted states degrading policy performance.

These findings suggest that applying RL to precision manipulation without prior structure (demonstrations, curriculum learning, or constrained action spaces) presents significant challenges. The thin contact margins of delicate labware create *extremely sparse* success conditions that pure exploration struggles to discover reliably. Future work will explore **demonstration-augmented RL** for initial behavioural scaffolding, **enlarged collision geometries** to ease contact detection, and **action clipping** to prevent simulation instabilities. Real-robot validation is planned to assess sim-to-real transfer. The ultimate goal remains making precision laboratory automation accessible to facilities that cannot afford traditional robotic systems.

2 Background

Laboratory automation could revolutionise bioengineering research, if only it weren't so expensive. Robotic arms precise enough for delicate lab work have traditionally cost **$50,000+**. But the hardware barrier has fallen: open-source designs like the **SO-101** now cost **under $300**.

Imagine a future where researchers spend their time on *experimental design* rather than pipetting, startups run overnight protocols autonomously, and protocols are shared as reproducible **robot policies** instead of ambiguous written instructions.

> *“This isn't about replacing scientists, but giving them back their time to do actual science.*
>
> THESIS

The remaining challenge is **training**. Unlike traditional imitation learning, which requires expensive human demonstrations, RL agents learn through *simulated trial and error* over millions of runs.

$300	$50k+	≈166×
SO-101 (OPEN HARDWARE)	INDUSTRIAL PRECISION ARM	COST REDUCTION

RESEARCH QUESTION

Can RL teach a low-cost robot to pick up a petri-dish lid without cracking, dropping, or disturbing the contents, using only *simulated trial and error*?

WHY SOFT ACTOR-CRITIC

I implemented **Soft Actor-Critic (SAC)**, an off-policy algorithm chosen over **TD3** because its *entropy maximisation* provides more principled exploration for contact-rich tasks. The stochastic policy should discover more robust approaches to handle variations in dish positioning.

As an **off-policy** method, SAC learns from a replay buffer of past experiences, more sample efficient than on-policy methods like **PPO**. As a **model-free** method, it learns directly from MuJoCo's simulation rather than building a secondary world model.

SAC UPDATE LOOP · PER GRADIENT STEP

1. **Entropy tuning:** α auto-adjusts to maintain target entropy
2. **Critic update:** twin Q-networks; minimum prediction reduces overestimation
3. **Actor update:** maximise $Q - \alpha \cdot \log \pi$ (reward + exploration)
4. **Target update:** Polyak averaging ($\tau = 0.005$) stabilises learning

TASK FORMULATION

- **Observation:** 21-D state vector. Joint positions (6), joint velocities (6), gripper position (3), Euler orientation (3), lid position (3)
- **Action:** 4-D. X/Y/Z end-effector displacement and a gripper open/close command
- **Success:** lid grasped, lifted to target height, held stable; dish base within 1 cm of its initial position

3 Methods

I forked Gota Gando's **pick-101** repository and adapted it for petri-dish manipulation, using his blog to guide design choices. The codebase uses the SAC implementation from **Stable-Baselines3**.

SIMULATION SETUP

The system uses **MuJoCo** (Google DeepMind), the **SO-101** model from The Robot Studio & Hugging Face, and a petri-dish mesh from GrabCAD which I modified to include collision primitives. Trained on a single RTX 4050 GPU.

Gando discovered that mesh-to-mesh collisions are unstable in MuJoCo and added box primitives to the gripper fingertips for stable contact. I built on this with invisible **cylinder primitives** for the petri dish: fingertips contact the cylinders while the visible mesh remains purely visual, improving collision stability.

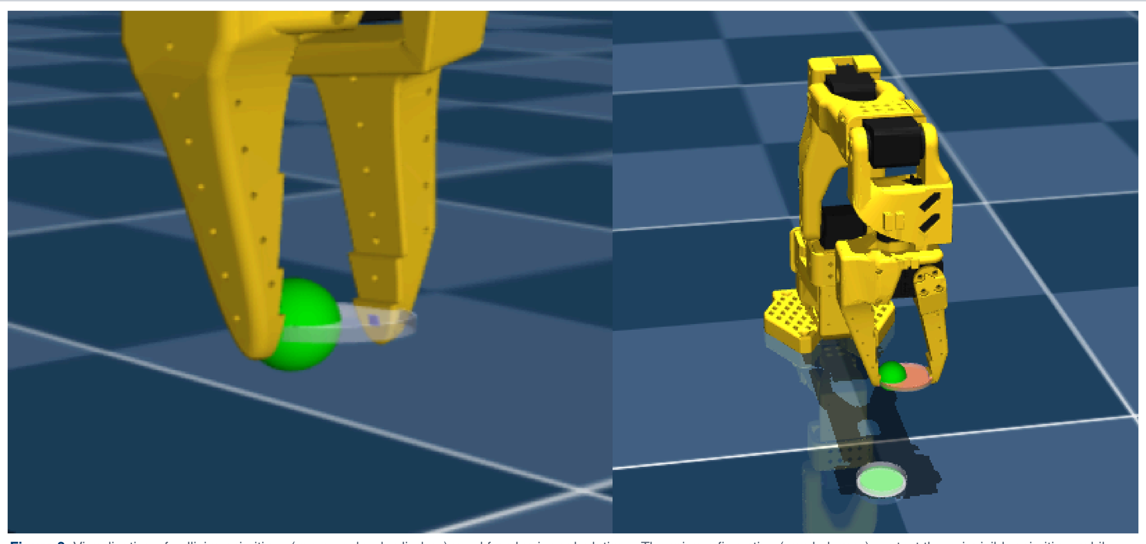


Figure 2. Visualisation of collision primitives (green and red cylinders) used for physics calculations. The gripper fingertips (purple boxes) contact these invisible primitives while the visible petri-dish mesh remains purely visual.

REWARD DESIGN (V19 → V20)

Following Gando, I started at **v20**, adapting his cube-lifting v19 to petri-dish mechanics:

- Height alignment to guide precise descent
- Contact bonuses for single & bilateral touch
- Closing incentive at correct height
- Gentle pressure with tolerance bounds
- Penalties for dish displacement & gripper jitter
- Increased grasp bonus, tighter thresholds

TRAINING PROTOCOL

- 500 k steps · 400 k buffer · batch 256 · $\gamma = 0.99$ · $\tau = 0.005$ · lr 3×10^{-4}
- 200–300 k steps (~2–3 h), terminated early on reward hacking; clean runs to 500 k (~5 h)
- Final setup used the **full task** from initialisation, which produced more robust policies than curriculum stages

4 Results

SAC explored effectively early but quickly stagnated, falling into reward hacking across all experiments. Across **15 reward iterations**, agents consistently exhibited one of two failure modes: **forceful manipulation** that risked damage, or **overly cautious hovering** in locally optimal positions.

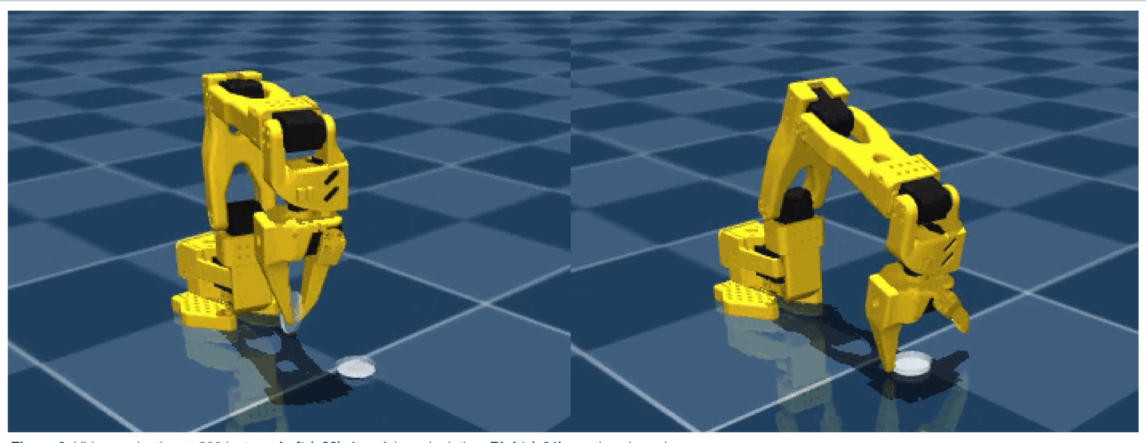


Figure 3. Video evaluation at 200 k steps. **Left (v22):** forceful manipulation. **Right (v31):** cautious hovering.

HEADLINE OUTCOME

1/30+	20%	150k
EXPERIMENTS THAT LIFTED THE LID	SUCCESS RATE (V22, FORCEFUL)	STEP AT WHICH RUNS PLATEAU

After each sparse success, performance **rapidly collapsed**. Evaluation reward converged to zero despite occasional lifts, suggesting the learned policy was **highly unreliable**. The agent kept hitting local optima due to the tiny margin of error.

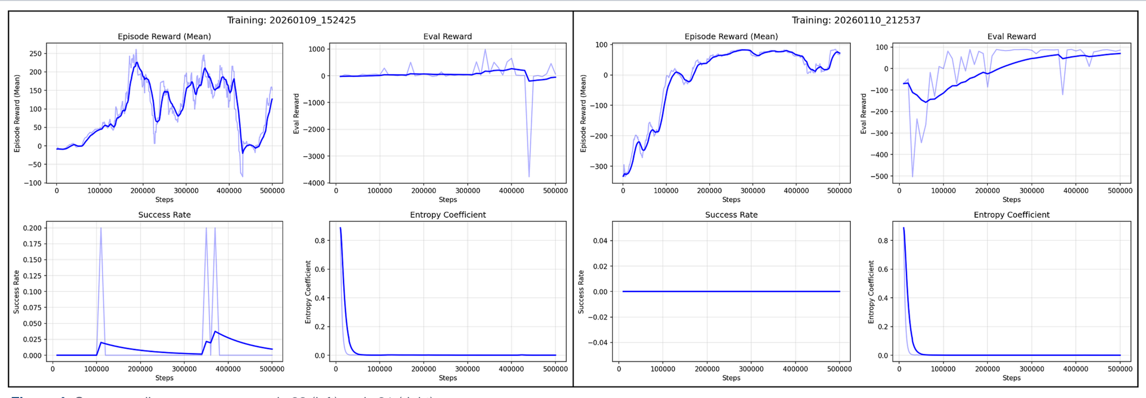
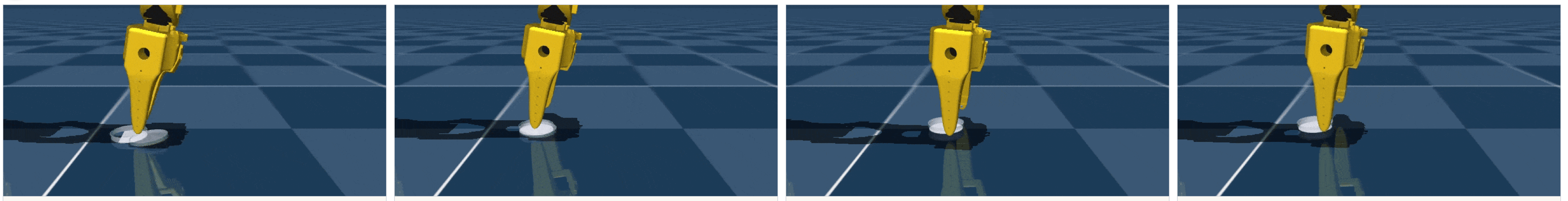


Figure 4. Corresponding curves on reward v22 (left) and v31 (right).

5 Reward hacking

REWARD VERSIONS V23 – V26



A
REWARD V23
Repeated closing without contact
Closes the gripper repeatedly *without contacting the lid*, triggering the closing-action reward in mid-air.
MITIGATION
Conditional rewards requiring maintained contact *during* the close.

B
REWARD V24
Lid-top stable contact
Establishes contact on the *lid top* but never closes, never lifts. Locally optimal hover earning contact reward indefinitely.
MITIGATION
Penalise top contact, reward skirt contact, escalate close & lift rewards.

C
REWARD V25
Lid-skirt stable contact
Maintains contact on the skirt but still refuses to close or lift, a deeper local optimum than (B), reached from correct geometry.
MITIGATION
Escalating closing reward, gated on skirt-contact maintenance.

D
REWARD V26
Skirt-contact jittering
Holds skirt contact while *oscillating* at high frequency to accumulate time-based reward without productive motion.
MITIGATION
Smoothness penalties on action variance; clip extreme commands.

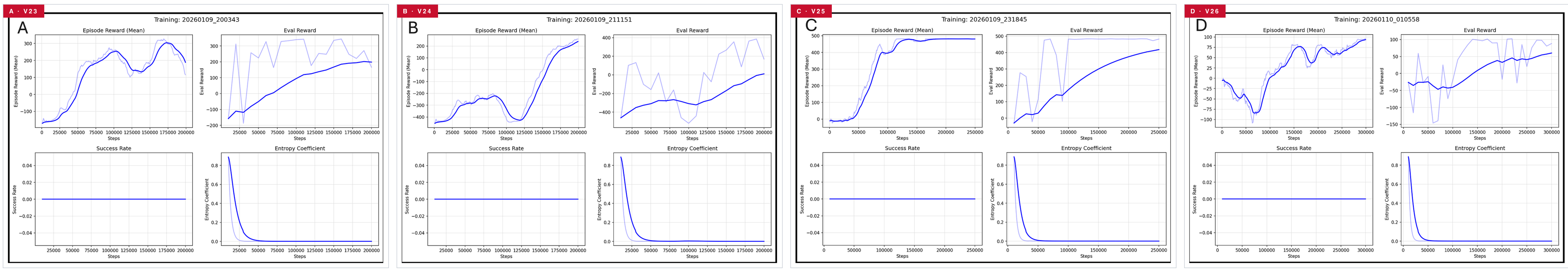


Figure 6. Corresponding training curves for (A) v23, (B) v24, (C) v25, (D) v26. Within each panel: episode reward (top-left), eval reward (top-right), success rate (bottom-left), entropy coefficient (bottom-right).

PATTERN · FIG 6 | *All attempts* **plateaued by 150 k steps** *with logarithmic convergence. Agent B's curve looked promising, but video evaluation revealed the same unproductive top-contact behaviour.* **Rising train reward ≠ task is being learned.** *Always watch the video.*

6 Conclusions

These results reveal both the **promise** and **fundamental challenges** of RL for delicate manipulation tasks. While v22 demonstrated that the task is learnable in principle, the low success rate, training instability, and forceful execution suggest significant barriers remain before practical deployment.

Reward shaping proved particularly challenging. In theory, a well-shaped reward leads to the optimal policy. In practice, effective rewards must:

- Guide the agent **without over-constraining**
- Ensure the **final goal dominates** intermediate rewards
- Accelerate **convergence** toward the optimal policy

The **simulation-to-reality gap** presents a key limitation: in real-world applications the robot would need visual or tactile inputs, while in simulation it received perfect state observation of the dish position and contact detection.

Despite this challenge, RL offers a **scalable approach** to laboratory automation. Beyond petri dishes, learned policies could handle **pipetting**, **sample transfer**, and **hazardous materials**, reducing accidents and minimising researcher exposure to dangerous substances.

> *“The potential impact on laboratory safety and productivity, combined with decreasing costs of both robotic hardware and computational resources, makes this a compelling direction for future research.*
>
> CLOSING REMARKS

7 Future directions

Future work should consider **enlarged collision geometry** or improved curriculum learning, pretraining agents on environments with reduced initial-state noise before introducing full task variability. For more reliable training or safe real-world application, implementing action limits and invalid-state detection would be essential.

- **Enlarged collision geometry:** the 1.25 mm fingertip vs. 1.5 cm lid is a strict contact-detection bottleneck (Fig 10)
- **Action clipping & early termination:** keep NaN/Inf states out of the replay buffer
- **Demonstration-augmented RL:** seed exploration with teleoperated lifts before SAC explores
- **Real SO-101 validation:** sim-to-real with vision and tactile inputs
- **Beyond petri dishes:** pipetting, sample transfer, hazardous materials

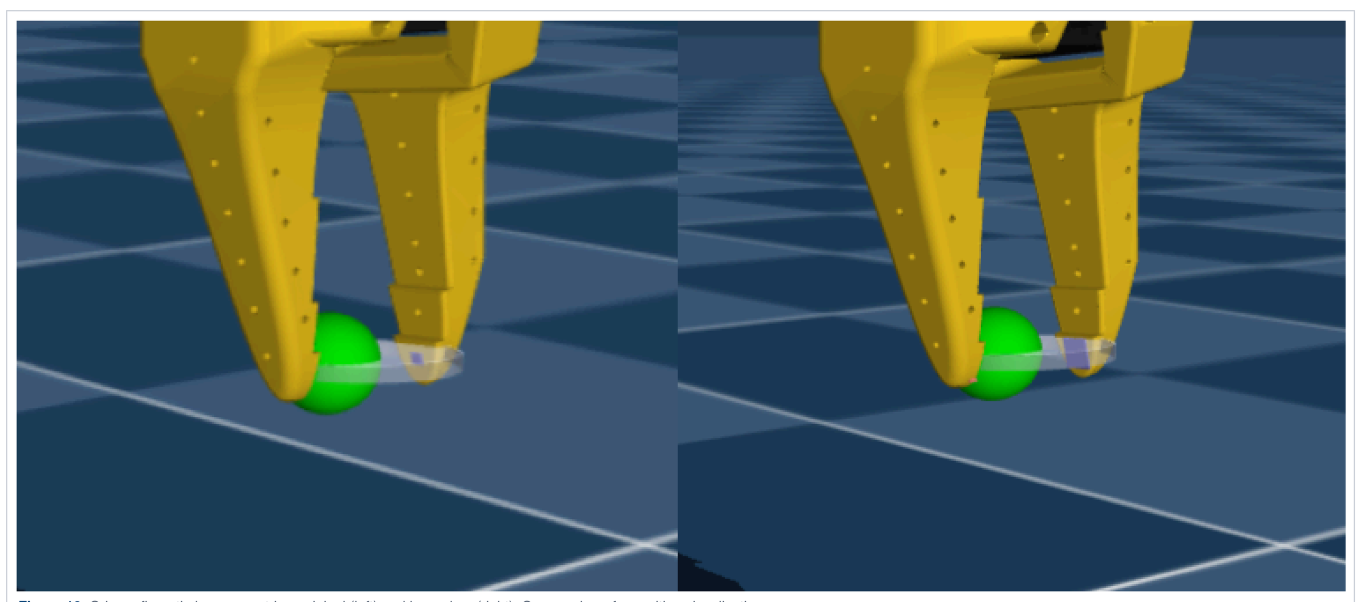


Figure 10. Gripper fingertip box geometries, original (left) and larger box (right). Green sphere for position visualisation.

READ THE LONG-FORM

Scan to read the full *article* on Medium.

Scan or visit
medium.com/@nataliefwc/democratising-lab-automation

REFERENCES
[1] Haarnoja, T. *et al.* (2018). Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning. *arXiv:1801.01290*
[2] Gando, G. *pick-101* — original cube-lifting RL framework. github.com/ggand0/pick-101
[3] The Robot Studio. SO-ARM100 / SO-101 open-source robotic arm. github.com/TheRobotStudio/SO-ARM100
[4] Todorov, E. *et al.* MuJoCo physics engine. github.com/google-deepmind/mujoco
[5] Raffin, A. *et al.* Stable-Baselines3 — SAC implementation. stable-baselines3.readthedocs.io
[6] GrabCAD. Petri dish 87 mm model. grabcad.com/library/petri-dish-87mm-1
[7] Gando, G. SO-101 RL blog. ggando.com/blog/so101-rl-lift
[8] Adapted code for petri-dish manipulation. github.com/snowcodeer/pick-101